Difference Between Frontend And Backend With Real Time Examples

The front end and back end are the two sides of the coin that make websites and applications function. Here's a breakdown to understand the difference with real-time examples:

Front End (Client-side):

- Imagine the front end as the storefront of a physical store. It's what you see and interact with directly.
- In a website, this includes elements like buttons, menus, text, images, and videos.
- Front-end developers focus on creating a user-friendly and visually appealing interface for users.
- **Example:** When you browse products on an e-commerce website, sort them by price, or add them to your cart, you're interacting with the front end. The code that controls the layout, animations, and responsiveness of these elements is all front-end development.

Back End (Server-side):

- Think of the back end as the stockroom and behind-the-scenes operations of the store. It's where all the unseen magic happens.
- In a website, this involves storing data (like product information, user accounts), processing user requests (like adding items to a cart), and interacting with databases.
- Back-end developers focus on the logic and functionality that make the website work.
- **Example:** When you click "add to cart" on the e-commerce website, the button sends a signal to the back end. The back end then updates your shopping cart in the database and sends a confirmation back to the front end to update the shopping cart display.

Here's a table summarizing the key differences:

Feature	Front End (Client-side)	Back End (Server-side)
Focus	User Interface (UI) and User Experience (UX)	Functionality and data management
Technologies	HTML, CSS, JavaScript	Programming languages like Java, Python
User Interaction	Direct interaction with elements	Indirect interaction through the front end

Export to Sheets

Analogy:

Imagine a restaurant. The menu you see and the waiter you interact with are the front end. The kitchen where the food is prepared and the storage room where the ingredients are kept are the back end. You don't see the back end directly, but it's what allows you to have a delicious meal!

why git? is there any other softwares and what is the difference between them

Why Git?

- **Distributed Version Control:** Unlike some centralized systems, Git makes a complete copy of the project history on every developer's machine. This allows you to work offline and collaborate seamlessly.

- **Branching Power:** Git excels at branching, where you can create isolated environments to work on new features without affecting the main project. This is crucial for managing complex projects and testing new ideas.

- **Speed and Efficiency:** Many Git operations happen locally, making them very fast. You can also revert to previous versions or compare changes easily.

- **Security and Data Integrity:** Git useschecksums to ensure data integrity, making it difficult to accidentally corrupt your project history.

- **Widely Used:** Being the most popular version control system (VCS), Git offers a vast ecosystem of tools, resources, and a large community for support.

About Version Control

What is “version control”, and why should you care? Version control is a system that records changes to a file or set of files over time so that you can recall specific versions later. For the examples in this book, you will use software source code as the files being version controlled, though in reality you can do this with nearly any type of file on a computer.

A version control system is a kind of software that helps the developer team to efficiently communicate and manage(track) all the changes that have been made to the source code along with the information like who made and what changes have been made.

Alternatives to Git:

While Git dominates the VCS landscape, there are alternatives that might suit specific needs:

- **Mercurial:** Similar to Git in terms of being distributed and open-source. It offers a simpler command-line interface but might lack some of Git's advanced branching features.

- **Subversion (SVN):** A centralized VCS that's easier to learn but less flexible than Git for branching and offline work. It might be a better fit for simpler projects with a linear development flow.

- **Fossil:** A standalone VCS that offers built-in bug tracking and wiki functionality. It can be a good choice for small teams who prefer a simpler setup.

is there any alternatives for git what is the difference between them

Yes, there are alternatives to Git, although Git is the dominant version control system (VCS) used today. Here's a breakdown of some alternatives and how they differ from Git:

Mercurial:

- Similarities: Open-source, distributed VCS like Git.
- Differences: Simpler command-line interface compared to Git. May lack some of Git's advanced branching features. Better suited for projects where ease of use is a priority.

Subversion (SVN):

- Similarities: Open-source VCS system.

- Differences: Centralized system, unlike Git's distributed model. Easier to learn due to its simpler structure. Less flexible for branching and offline work. Might be a better choice for linear development workflows in smaller teams.

Fossil:

- Similarities: Open-source VCS.
- Differences: Standalone system that integrates bug tracking and wiki functionalities. Easier setup for smaller teams who don't need a complex external server setup. Might not be ideal for large-scale projects.

Choosing the Right VCS:

- **Team familiarity:** If your team is already comfortable with Git, there's little reason to switch unless there's a specific need.
- **Project complexity:** Git's strengths lie in handling complex projects with frequent branching and merging.
- **Ease of use:** For smaller teams or simpler projects, Subversion or Fossil might be easier to learn and use.

Here's a table summarizing the key differences:

Feature	Git	Mercurial	Subversion (SVN)	Fossil
Model	Distributed	Distributed	Centralized	Standalone
Learning Curve	Steeper	Moderate	Easier	Easier
Branching Features	Powerful	Moderate	Limited	Limited
Offline Work	Supported	Supported	Not ideal	Supported
Community & Resources	Vast	Large	Large	Moderate

Export to Sheets

Ultimately, the best VCS depends on your specific project needs and workflow. Consider the factors mentioned above to make an informed decision.

GIT Fork

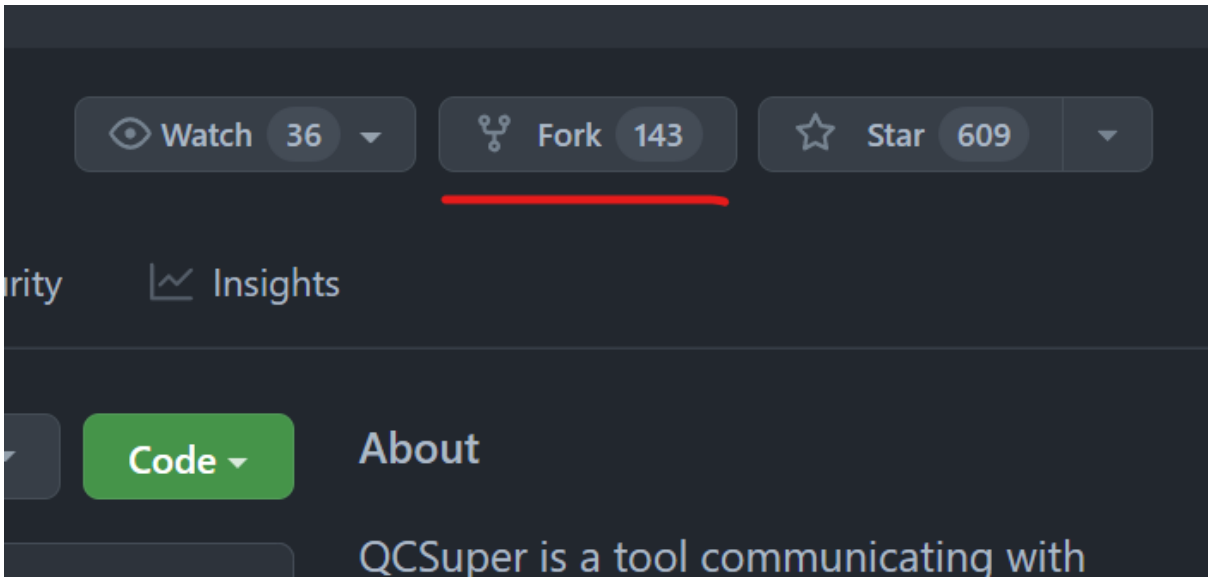
Forking a repository means creating a copy of the repo. When you fork a repo, you create your own copy of the repo on your GitHub account. When several developers want to work on a project but need to make changes that are inappropriate for the original repository, forking is frequently used in open-source software development.

Reasons For Forks

- You have your own copy of the project on which you may test your own changes without changing the original project.
- This helps the maintainer of the project to better check the changes you made to the project and has the power to either accept, reject or suggest something.
- When you clone an Open-Source project, that isn't yours, you don't have the right to push code directly into the project.

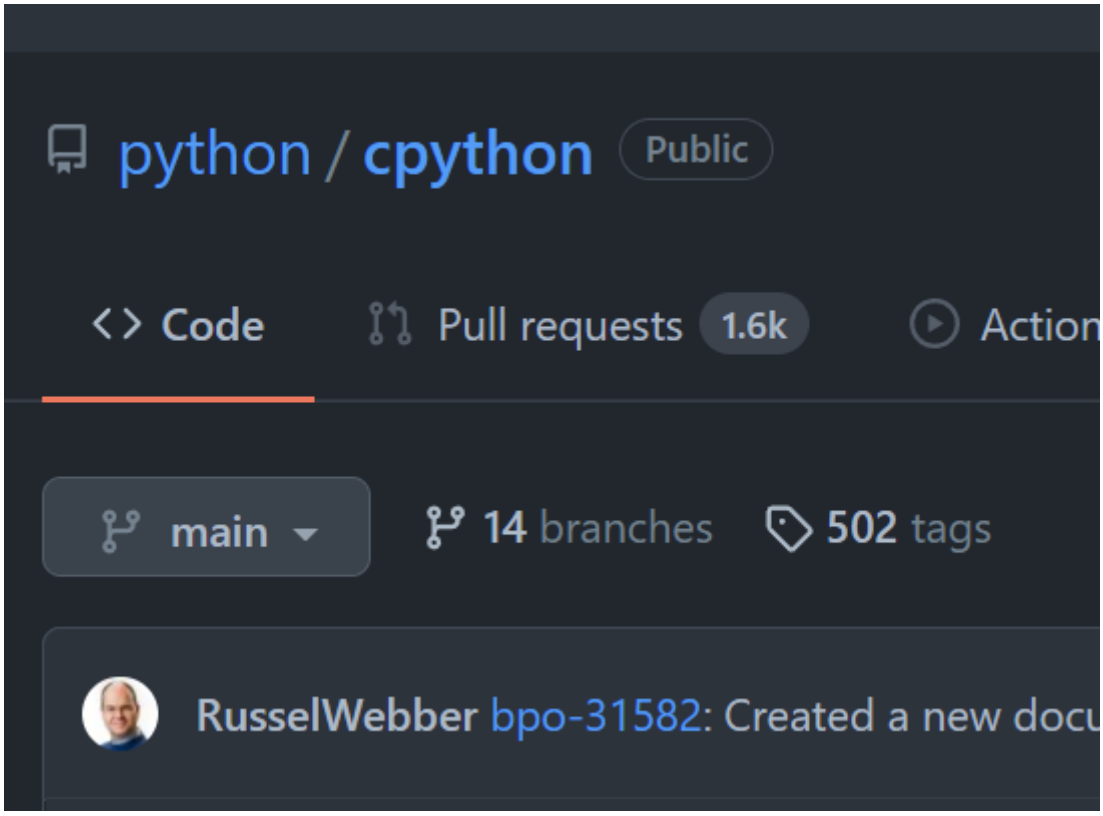
Steps to Forking A Repository

Step 1: Open the repository that you want to Fork there You can see the icon as shown in the image below in the repo's top right corner. Now, this feature is used to Fork the repo.



Git Fork

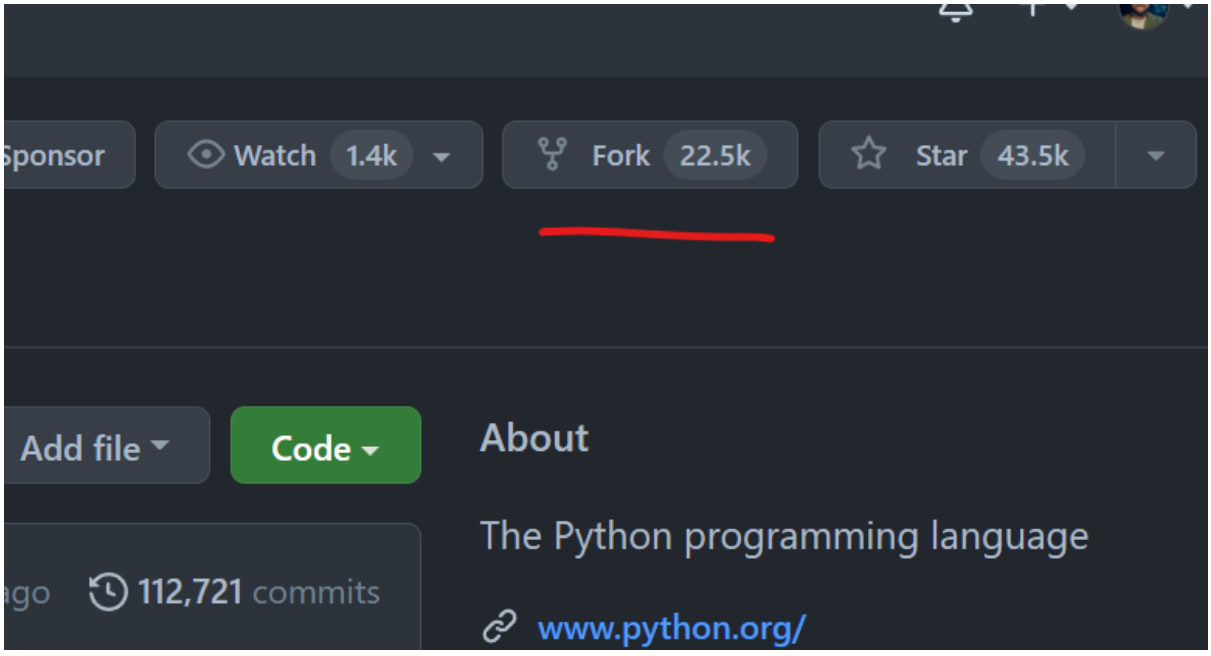
Step 2: Go to any repository that you want to Fork here we are using a sample repo of **Python** official repository.



Git Repo

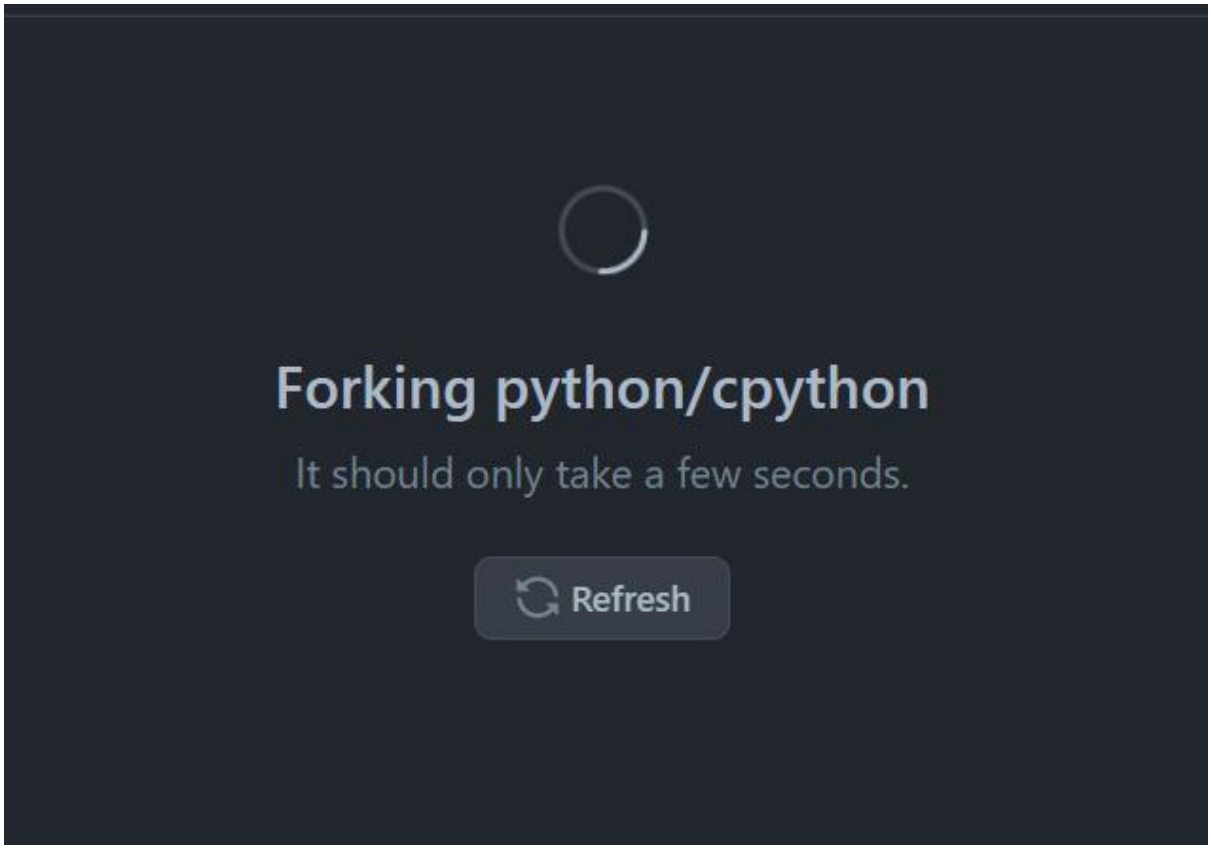
You can see **python/cpython**. This means python is the maintainer and cpython is the project's name.

Step 3: Find the Fork button in the top right corner.



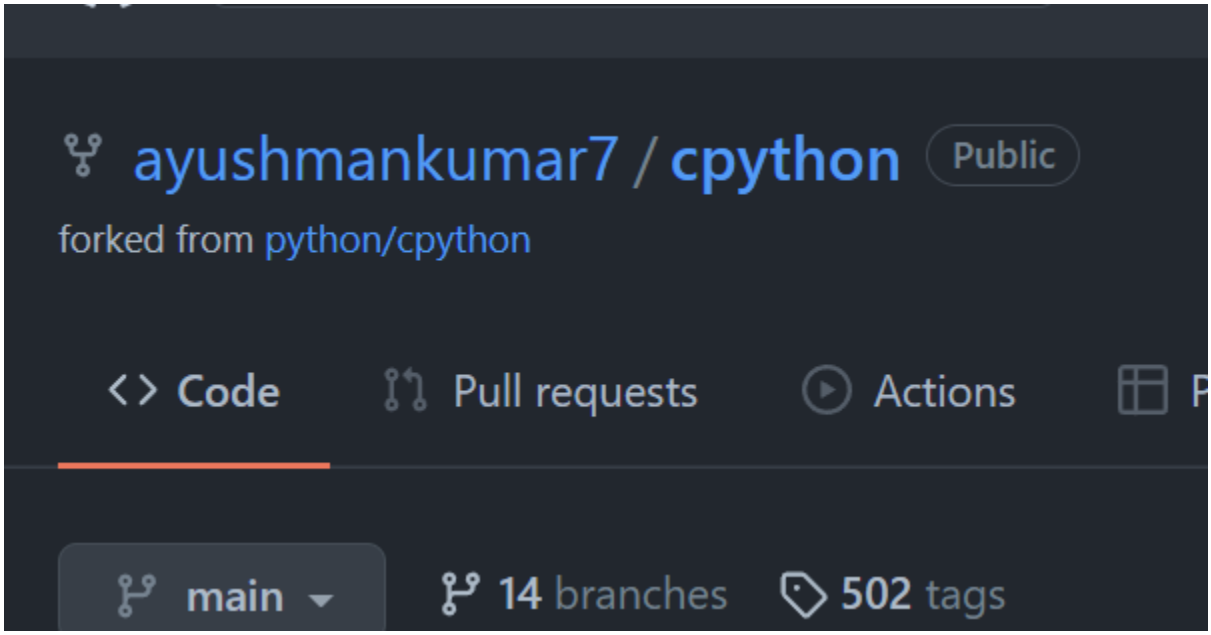
Fork button

Step 4: Click on **Fork**.



Forking a repo

Step 5: Now you have your own copy of the repository. But how can we confirm for which do refer to below visual aid as follows:



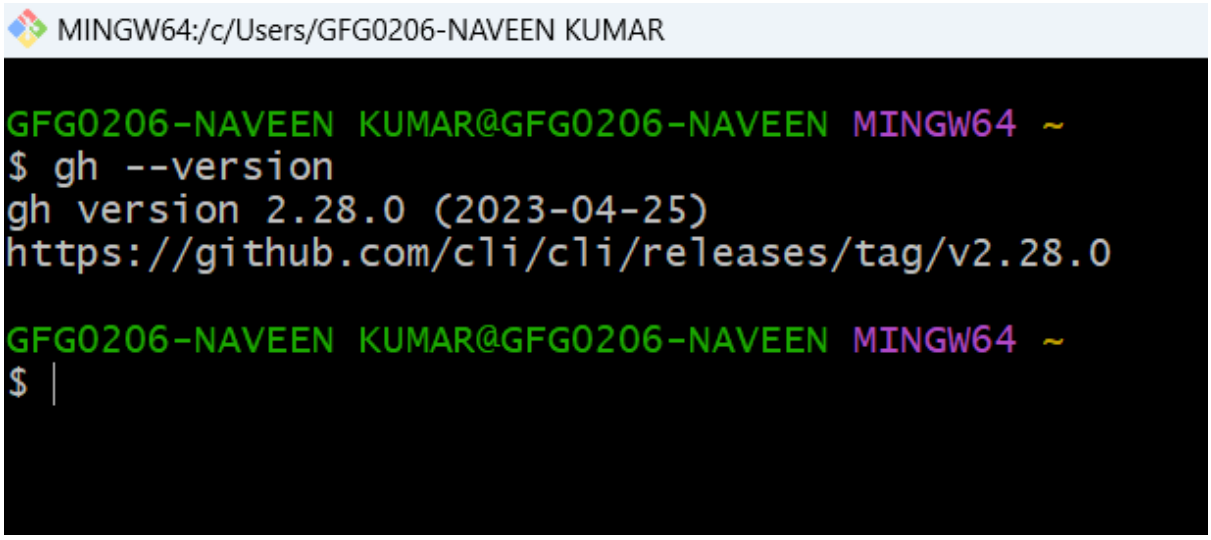
Now we can see your user name(**********)/CPython and also below that, we have the link to the original project I forked from.
Whatever changes are made to '******/python? We can make my changes here and then make a Pull Request to the maintainers of the project. Now it is in their hand if they will accept or reject your changes to the main project.

Fork Using Command Line

Step 1: Open the terminal or gitbash and type the below command.

```
gh --version
```

If it displays the version like in the image below, then the GitHub Cll was already set up on our computer. If not, use this link to set it up. "https://cli.github.com/"

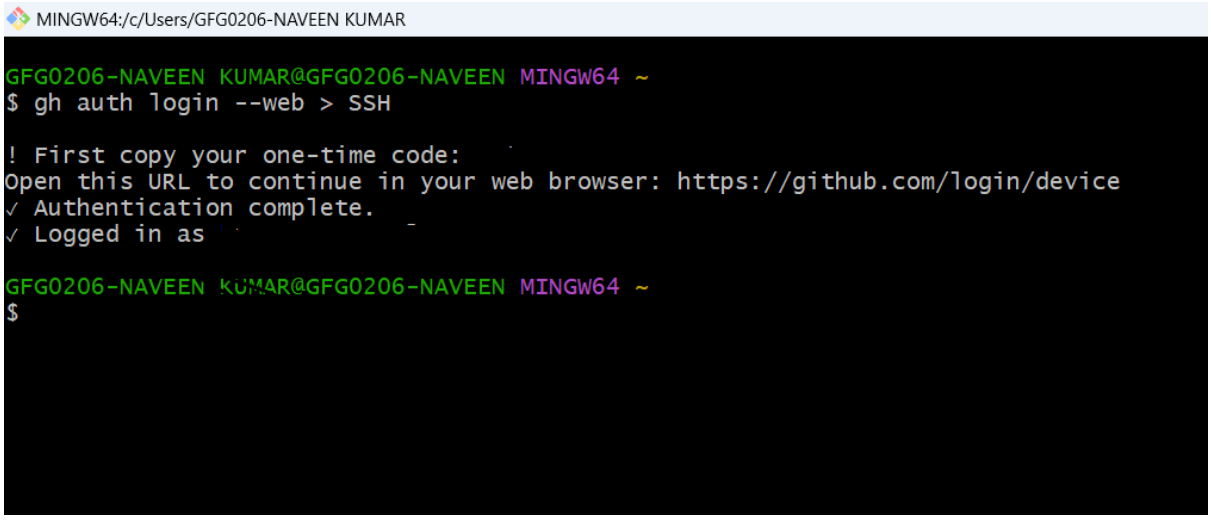


GitBash Version

Step 2: You must first log in using the CLI to GitHub using your GitHub account before you may fork the repositories. To do it, issue the following command.

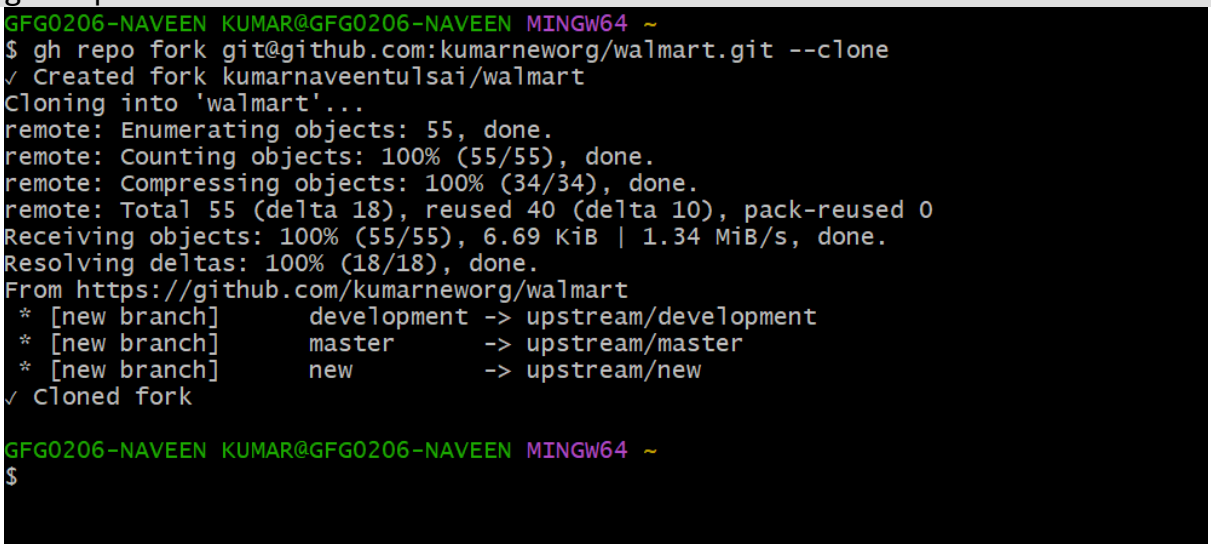
```
gh auth login --web > SSH
```

Open the CLI-provided link, then enter your one-time code in the Chrome browser.



Step 3: Once authentication is done. Copy the Repo URL that you to fork into our repo and use the below command.

```
gh repo fork <REPO URL> --clone
```



Fork the repo

— clone is used to get rid of the hurdles in between Fork. That is how we may use the CLI to fork the repository.

Difference between Git Fork vs Git Clone

Fork	Clone
To enable independent development without impacting the original repository, a fork is a duplicate of a repository that has been made.	Cloning a repository entails making a local duplicate of an already-existing, remotely hosted Git repository.

Fork	Clone
When you fork a repository, a new copy of the repository is created under your own account enabling you to experiment and edit as necessary.	A repository is completely duplicated when it is cloned, including all of its files, history, and metadata.
A forking operation produces a fresh, independent copy of the repository that belongs to the forking user and is commonly carried out on a Git hosting service like GitHub or GitLab.	Cloning is commonly carried out via the Git command line on a local system, and it produces an isolated, complete copy of the repository that may be edited and committed locally.

How To Decide Between Git Clone And Fork?

It is advised to utilize **“git clone”** if you are employed by an organization as a developer and you wish to make contributions to the source code., which is better suited for the **“git push and git pull”** operations via which other developers can view and validate the code.

Use **“git fork”** so other developers can’t access or follow your progress if you wish to construct another project in isolation using existing source code as a base.